YOLO Object Detection

Train and deploy a smoke and fire detector, step by step

This tutorial takes you from the basic ideas of object detection to a working web app. You will train an Ultralytics YOLO11 model on the D-Fire dataset using a free Kaggle GPU, evaluate it, and serve it through a FastAPI backend with a browser interface. Every code block can be copied into a Kaggle notebook or a Python file as written.

You need	Details
Kaggle account	Phone-verified (required to turn on GPU and Internet)
Python basics	Running cells, reading tracebacks, installing packages with pip
Local machine (for the web app)	Python 3.9+; any OS. No GPU needed for single-image inference
Time	About 1 hour of setup and reading, plus up to 3 hours of GPU training

Contents

1. Object detection basics 2

2. Prepare Kaggle and the dataset 3

3. Train the model 5

4. Evaluate and improve 7

5. Save and export the model 8

6. Build the web app 8

7. Troubleshooting 11

8. Limits and responsible use 11

9. Exercises 11

10. Glossary 12

1. Object detection basics

Computer-vision models answer different questions about an image:

Task	Question it answers	Output
Classification	What is in this image?	One label (e.g. fire)
Object detection	What is where? (this tutorial)	A box, a class and a confidence score for each object
Segmentation	Which exact pixels belong to each object?	A mask per object

Detection suits smoke and fire because an image can contain none, one or several, in different places, and you want to know where.

1.1 How YOLO works

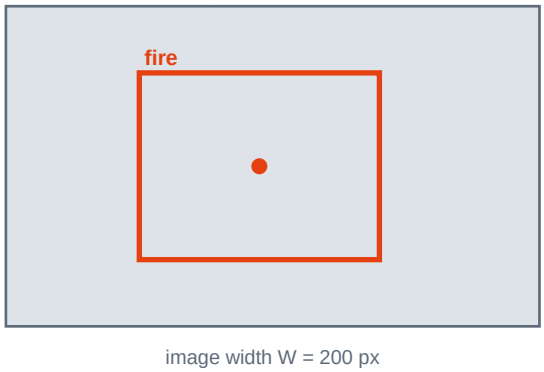
YOLO stands for *You Only Look Once*. A single neural network looks at the whole image in one pass and predicts many candidate boxes, each with a class and a confidence. It then removes duplicates with **non-maximum suppression (NMS)**, which keeps the best box among heavily overlapping ones. The result is fast enough for video.

- **Backbone**: extracts visual features at several scales.
- **Neck**: mixes those features so small and large objects are both seen.
- **Head**: outputs the boxes, class scores and confidences.
- **Pretrained weights** (yolo11s.pt) were trained on the COCO dataset's 80 classes. When you train on your own two classes you keep the backbone and neck knowledge and retrain the head. This is transfer learning, and it is why a few thousand images are enough.

1.2 The YOLO label format

Each image has a text file with the same name. Each line describes one object as five numbers, all normalized to the 0-1 range:

```
class_id  x_center  y_center  width  height
```



Label line for this box:

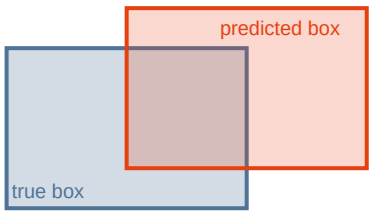
1 0.475 0.500 0.450 0.583

```
class_id = 1 (fire)
x_center = 95 / 200 = 0.475
y_center = 60 / 120 = 0.500
width = 90 / 200 = 0.450
height = 70 / 120 = 0.583
```

Centre and size are fractions of the image, so labels stay valid if the image is resized.

An image with no objects has an **empty** label file. These are called background images and they teach the model what is not smoke or fire.

1.3 Measuring quality



IoU = overlap area / combined area

IoU = 1.0: perfect match. IoU = 0: no overlap.

mAP50 counts a hit when IoU >= 0.5.

mAP50-95 averages thresholds 0.50, 0.55 ... 0.95.

Metric	Meaning
Precision	Of everything the model detected, how much was correct (few false alarms = high precision)
Recall	Of everything really there, how much was found (few misses = high recall)
Confidence	The model's own score for one box. You choose a threshold (conf) below which boxes are hidden
AP	Average precision for one class, across all confidence thresholds
mAP50	Mean of AP over classes, counting a hit when IoU >= 0.5
mAP50-95	Mean over IoU thresholds 0.5 to 0.95: a stricter, lower number. The standard headline metric

1.4 Model sizes

Variant	File	Trade-off
Nano	yolo11n.pt	Fastest, smallest, least accurate. Good for quick tests and CPUs
Small	yolo11s.pt	Balanced. Used in this tutorial
Medium	yolo11m.pt	More accurate, slower; needs more GPU memory
Large / X	yolo11l.pt, yolo11x.pt	Highest accuracy, slowest

> **Note**
> Newer Ultralytics releases may ship newer weights (for example yolo26*.pt). The code in this tutorial is unchanged: only the weights file name changes.

2. Prepare Kaggle and the dataset

2.1 Notebook settings

- Create a new notebook at kaggle.com. Verify your phone number in your account settings first.
- Open **Session options** (right sidebar) and set **Accelerator = GPU T4** (or P100). Turn **Internet = On**.
- Kaggle gives a limited weekly GPU quota (roughly 30 hours; check your account page). Sessions also time out after several hours, so long runs need a time cap and resume (section 4.3).

```
import torch
print(torch.cuda.is_available(), torch.cuda.device_count())   # expect: True 1  (or 2)
!pip install -q ultralytics
!yolo settings wandb=False      # stops the Weights & Biases login prompt
```

> **Watch out**
> If this prints **False 0**, no GPU is attached. Training would fail with 'Invalid CUDA device' or crawl on the CPU. Fix the accelerator setting and restart the session before going further.

2.2 Add the dataset and inspect it

Use **Add Input** and search for *smoke fire detection yolo* (author sayedgamal99), or paste its path. Datasets are mounted read-only under /kaggle/input. Listing a huge dataset with os.walk is slow because it visits every file, so list only the top levels:

```
import os
SRC = "/kaggle/input/datasets/sayedgamal99/smoke-fire-detection-yolo"

def show(path, depth=0, max_depth=3):
    if depth > max_depth: return
    with os.scandir(path) as it: entries = list(it)
    dirs = [e for e in entries if e.is_dir()]
    print("  " * depth + os.path.basename(path), f"| {len(entries) - len(dirs)} files")
    for d in dirs: show(d.path, depth + 1, max_depth)

show(SRC)
```

Expected structure (D-Fire):

```
smoke-fire-detection-yolo/
  data.yaml                <- original config (names: ['smoke', 'fire'])
  data/
    train/images  14122 files    train/labels  14122 files
    val/images     3099 files    val/labels     3099 files
    test/images    4306 files    test/labels    4306 files
```

2.3 Copy the data to local disk

Reading from /kaggle/input is very slow (under 1 MB/s was measured here), and Ultralytics also wants a writable folder for its label cache. Copy the data to /kaggle/working once per session. A single cp -r is slow and easy to interrupt, so use threads. This version is **resumable**: it skips files already copied, and finishes with a count check.

```
import os, shutil
from concurrent.futures import ThreadPoolExecutor

SRC = "/kaggle/input/datasets/sayedgamal99/smoke-fire-detection-yolo/data"
DST = "/kaggle/working/dataset"

jobs = []
for s in ["train", "val", "test"]:
    for t in ["images", "labels"]:
        os.makedirs(f"{DST}/{s}/{t}", exist_ok=True)
        for f in os.listdir(f"{SRC}/{s}/{t}"):
            d = f"{DST}/{s}/{t}/{f}"
            if not os.path.exists(d):
                jobs.append((f"{SRC}/{s}/{t}/{f}", d))
print("files to copy:", len(jobs))

def cp(p):
    try: shutil.copyfile(*p)
    except Exception: return p[0]

with ThreadPoolExecutor(32) as ex:
    failed = [r for r in ex.map(cp, jobs) if r]
print("failed:", len(failed))

for s, n in [("train", 14122), ("val", 3099), ("test", 4306)]:
    for t in ["images", "labels"]:
        got = len(os.listdir(f"{DST}/{s}/{t}"))
        print(s, t, got, "OK" if got == n else "INCOMPLETE")
```

> **Watch out**
> Do not stop the cell or close the tab while it runs. If a count says INCOMPLETE, just run the cell again. Symlinks do not help: they still read from the slow input folder. A new session wipes /kaggle/working, so repeat this step then.

2.4 Write data.yaml

This file tells YOLO where the images are and what the classes are called. Class order must match the labels (0 = smoke, 1 = fire).

```
import yaml
yaml.dump({"path": "/kaggle/working/dataset",       # must be the LOCAL copy
           "train": "train/images", "val": "val/images", "test": "test/images",
           "names": {0: "smoke", 1: "fire"}},
          open("/kaggle/working/data.yaml", "w"))
print(open("/kaggle/working/data.yaml").read())
```

2.5 Sanity-check the labels

Always look at your data before training. First confirm the class IDs actually used, then draw one labelled image.

```
import collections, os
c = collections.Counter()
for f in os.listdir(f"{DST}/val/labels")[:500]:
    for line in open(f"{DST}/val/labels/{f}"):
        if line.strip(): c[line.split()[0]] += 1
print(c)        # keys '0' (smoke) and '1' (fire) only; compare the sizes of the two

import cv2, glob, matplotlib.pyplot as plt
img_path = sorted(glob.glob(f"{DST}/train/images/*"))[100]
lbl_path = img_path.replace("/images/", "/labels/").rsplit(".", 1)[0] + ".txt"
img = cv2.cvtColor(cv2.imread(img_path), cv2.COLOR_BGR2RGB)
h, w = img.shape[:2]
for line in open(lbl_path):
    cls, xc, yc, bw, bh = map(float, line.split())
    x1, y1 = int((xc - bw / 2) * w), int((yc - bh / 2) * h)
    x2, y2 = int((xc + bw / 2) * w), int((yc + bh / 2) * h)
    cv2.rectangle(img, (x1, y1), (x2, y2), (255, 80, 0) if cls == 1 else (80, 120, 200), 3)
plt.figure(figsize=(7, 5)); plt.imshow(img); plt.axis("off")
```

> **Tip**
> If the boxes sit in the wrong place, or the class counts look strange, fix the data now. Bad labels are the number one reason for low accuracy.

3. Train the model

```
from ultralytics import YOLO

model = YOLO("yolo11s.pt")          # downloads pretrained weights (Internet must be On)
model.train(
    data="/kaggle/working/data.yaml",
    epochs=30,
    time=3,                         # hours cap; overrides epochs
    imgsz=512,
    batch=32,
    device=0,                       # one GPU
    workers=4,
    cache="ram",                    # use "disk" if you run out of memory
    patience=10,
    plots=False,
    project="/kaggle/working/runs",
    name="smoke_fire",
    exist_ok=True,                  # reruns reuse the same folder name
)
```

3.1 What each setting does

Setting	Meaning and how to choose
data	Path to data.yaml
epochs	One epoch = one pass over all training images. More epochs give the model more chances to learn, up to the point of overfitting
time	Wall-clock cap in hours. If set, it **overrides epochs**: Ultralytics rescales the epoch count after the first epoch so the run, including its learning-rate schedule, fits the time
imgsz	Images are resized so the long side is this many pixels. 640 is the usual default; 512 is faster. Small objects need larger values. Use the same value later for prediction
batch	Images per step. Higher is faster but needs more GPU memory. Lower it (16, 8) on a memory error
device	0 = first GPU; [0,1] = two GPUs. Single GPU is simpler and avoids multi-GPU problems in notebooks
workers	CPU threads that load images. Kaggle has few cores, so 2-4
cache	Store decoded images in RAM or on disk so each epoch skips JPEG decoding
patience	Early stopping: quit if validation fitness has not improved for this many epochs
plots	False skips chart generation; results.csv is still saved
name / exist_ok	Output goes to project/name. Without exist_ok, a rerun makes name2, name3 ...

3.2 Reading the training log

Each epoch prints a progress bar with losses, then a validation line:

Column	Reading it
box_loss	How far predicted boxes are from true boxes. Should fall
cls_loss	Classification error (smoke vs fire vs background). Should fall
dfl_loss	Box-edge precision term. Should fall
P, R	Precision and recall on the validation set
mAP50, mAP50-95	Main quality scores on validation. Should rise, then flatten

What to expect: epoch 1 is slow because labels are scanned and images cached. Later epochs are faster. Healthy training shows falling losses and rising mAP. If training loss keeps falling while validation mAP stalls or drops, the model is **overfitting** (memorizing training images): early stopping via patience handles this.

3.3 Long runs: time cap, disconnects and resume

```
# if the notebook stops or disconnects, continue where it left off
model = YOLO("/kaggle/working/runs/smoke_fire/weights/last.pt")
model.train(resume=True)
```

Two files are written under `runs/smoke_fire/weights/`: **last.pt** (newest epoch, used for resume) and **best.pt** (the epoch with the best validation fitness, used for evaluation and deployment).

3.4 Inspecting results.csv

```
import pandas as pd
df = pd.read_csv("/kaggle/working/runs/smoke_fire/results.csv")
df.columns = df.columns.str.strip()
print(df.columns.tolist())                                  # check the exact column names
print(df[["epoch", "train/box_loss", "val/box_loss",
          "metrics/mAP50(B)", "metrics/mAP50-95(B)"]].tail(10))
```

> **Note**
> The 21 images reported as 'corrupt' with a Read-only file system error in the first scan are not truly corrupt. Ultralytics tried to repair them inside the read-only input folder. With the data on /kaggle/working this message disappears.

4. Evaluate and improve

4.1 Score on the test split

The test images were never used for training or to choose best.pt, so they give an honest estimate. Without `split='test'` the validation images are used instead.

```
import glob
best = sorted(glob.glob("/kaggle/working/runs/**/best.pt", recursive=True))[-1]
print("using:", best)       # make sure this is your newest run

model = YOLO(best)
m = model.val(data="/kaggle/working/data.yaml", split="test", imgsz=512, plots=True)
print("mAP50:", m.box.map50, "mAP50-95:", m.box.map)
print("per-class mAP50-95 [smoke, fire]:", m.box.maps)

from IPython.display import Image, display
display(Image(f"{m.save_dir}/confusion_matrix.png"))
```

The confusion matrix shows, for each true class, what the model predicted. Look at the *background* row and column: fires or smoke the model missed appear as predicted background, and false alarms appear as true background.

4.2 Look at real predictions

```
import glob
from IPython.display import Image, display
imgs = sorted(glob.glob("/kaggle/working/dataset/test/images/*"))[:8]
model.predict(imgs, conf=0.3, imgsz=512, save=True, project="/kaggle/working/runs", name="preds", exist_ok=True)
for p in sorted(glob.glob("/kaggle/working/runs/preds/*.jpg")): display(Image(p))

# video test: writes an annotated copy under runs/
model.predict("YOUR_VIDEO.mp4", conf=0.3, imgsz=512, save=True)

# simple alert rule: only alarm after 3 consecutive frames with fire
streak = 0
for r in model.predict("YOUR_VIDEO.mp4", conf=0.3, imgsz=512, stream=True):
    has_fire = any(r.names[int(b.cls)] == "fire" for b in r.boxes)
    streak = streak + 1 if has_fire else 0
    if streak >= 3:
        print("FIRE ALERT")
```

4.3 Improving a weak model

Symptom	Likely cause	What to try
Low mAP for everything	Label errors, too few epochs, model too small	Re-check labels (2.5); train longer; try yolo11m; imgsz=640
Smoke much lower than fire	Smoke is diffuse with soft edges	More and varied smoke images; longer training; larger imgsz
False alarms (clouds, fog, sunsets, orange lights)	Not enough hard negatives	Add background images of those scenes (empty label files); raise conf to 0.4-0.5
Missed detections	Threshold too high; small objects; little data	Lower conf; larger imgsz; more examples
Validation mAP falls while training loss falls	Overfitting	More data or augmentation; keep patience; fewer epochs
Out of memory	Batch or cache too large	Lower batch; cache='disk'; smaller model

> **Tip**
> Change one thing at a time and keep notes of the test mAP after each run, so you know what helped. For a quick experiment add fraction=0.3 (30% of the data) and use yolo11n.pt.

5. Save and export the model

5.1 Get your weights out of Kaggle

```
import shutil
from IPython.display import FileLink
shutil.copy(best, "/kaggle/working/best.pt")
FileLink("best.pt")             # click the link to download
```

Alternatively use **Save Version, then Save & Run All**; files in /kaggle/working appear in the notebook's Output tab. Note that Save & Run All re-runs the whole notebook, including training, so keep it for a final run.

5.2 Export versus save

Training already saves best.pt and last.pt. `model.export()` is not a save: it converts the model to a different format for other runtimes. It is optional.

Where it will run	File	Command
Python server (this tutorial)	best.pt	nothing to do
No PyTorch: C++, C#, JS, OpenCV	best.onnx	model.export(format='onnx', imgsz=512, simplify=True)
NVIDIA GPU, fastest	best.engine	model.export(format='engine')
Android / iOS / edge	best.tflite	model.export(format='tflite')

A model loaded from an exported file still works through Ultralytics: `YOLO('best.onnx').predict(...)`. Without Ultralytics you must do the resizing, normalization and NMS yourself.

6. Build the web app

Architecture: the browser sends an image to the server; the server runs YOLO and returns detections as JSON; the browser draws the boxes.

```
Browser (index.html)  --POST /detect (image + conf)-->  FastAPI (main.py + best.pt)
Browser draws boxes   <--JSON list of detections------  model.predict()
```

6.1 Project folder

```
smoke-fire-app/
  main.py          server
  index.html       frontend (the file delivered with this tutorial)
  best.pt          your trained weights
  requirements.txt
```

6.2 The FastAPI server

```
import io
from fastapi import FastAPI, UploadFile, File
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import HTMLResponse
from PIL import Image
from ultralytics import YOLO

app = FastAPI()
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"])
model = YOLO("best.pt")                     # loaded ONCE at startup, not per request

@app.post("/detect")
async def detect(file: UploadFile = File(...), conf: float = 0.3):
    img = Image.open(io.BytesIO(await file.read())).convert("RGB")
    r = model.predict(img, conf=conf, imgsz=512)[0]
    return [
        {"class": r.names[int(b.cls)], "conf": round(float(b.conf), 3),
         "box": [round(v, 1) for v in b.xyxy[0].tolist()]}      # x1, y1, x2, y2 in pixels
        for b in r.boxes
    ]

@app.get("/", response_class=HTMLResponse)
def home():
    return open("index.html", encoding="utf-8").read()
```

The `encoding='utf-8'` argument matters: on Windows, open() defaults to a legacy encoding and garbles characters such as the apostrophe and ellipsis in the page.

```
pip install fastapi uvicorn python-multipart ultralytics pillow
uvicorn main:app --reload                # open http://localhost:8000
```

Test the API without the page. FastAPI also generates interactive docs at `/docs`, where you can upload an image and try the endpoint.

```
curl -X POST -F "file=@test.jpg" "http://localhost:8000/detect?conf=0.3"
# [{"class":"fire","conf":0.87,"box":[120.5,80.2,340.0,290.7]}]
```

6.3 The frontend (index.html)

The page is one self-contained file. Open it through the server (http://localhost:8000), not by double-clicking, so the browser can call /detect and the camera is allowed.

Feature	How it works
Upload / drop / paste	File goes to /detect as multipart form data; the response is drawn on a canvas over the original image
Boxes	Ember red for fire, steel blue for smoke, with a confidence label. Line width scales with image size
Sensitivity slider	Sends conf as a query parameter and re-runs the last image
Live camera	Captures one frame per second (not full video rate) so the server is not flooded
Fire confirmation	In camera mode a fire shows as confirmed only after 3 consecutive frames, which reduces flashes from sunsets and lamps

Feature	How it works
Detector address field	Defaults to /detect; set a full URL if the page and API are on different hosts

The core of the frontend is just this request and a drawing loop:

```
async function detect(blob) {
  const fd = new FormData();
  fd.append("file", blob, "frame.jpg");
  const r = await fetch(`/detect?conf=${conf}`, { method: "POST", body: fd });
  return r.json();                       // [{class, conf, box:[x1,y1,x2,y2]}, ...]
}
// for each detection: ctx.strokeRect(x1, y1, x2 - x1, y2 - y1) and draw the label
```

> **Watch out**
> Browsers allow camera access only on **https** or **http://localhost**. On a plain http address (for example a LAN IP) the camera is blocked. For a deployed site use https, which most hosts provide automatically.

6.4 Quick alternative: Gradio on Hugging Face Spaces

If you only need a shareable demo, create a Gradio Space and upload best.pt, app.py and requirements.txt (ultralytics, gradio, opencv-python-headless). The free tier is CPU-only: fine for single photos, slow for video.

```
import gradio as gr
from ultralytics import YOLO

model = YOLO("best.pt")

def detect(img, conf):
    r = model.predict(img, conf=conf, imgsz=512)[0]
    return r.plot()[..., ::-1]           # BGR -> RGB for display

gr.Interface(fn=detect,
             inputs=[gr.Image(type="numpy"), gr.Slider(0.1, 0.9, 0.3, label="Confidence")],
             outputs=gr.Image(), title="Smoke & Fire Detection").launch()
```

6.5 Hosting the FastAPI app

Package it in a container and run it on a host such as Render, Railway, a cloud VM or any server that runs Docker. A starting point:

```
# requirements.txt
fastapi
uvicorn
python-multipart
ultralytics
pillow
opencv-python-headless

# Dockerfile
FROM python:3.11-slim
RUN apt-get update && apt-get install -y libgl1 libglib2.0-0 && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY main.py index.html best.pt ./
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

- The image is large because PyTorch is included. A CPU host handles roughly one image per fraction of a second to a second with yolo11s; a GPU host is faster.
- Replace `allow_origins=['*']` with your real site address before going public.
- Limit upload size and add authentication if the endpoint is public, to prevent abuse.
- Keep the model loaded in memory (as above). Loading it on every request would be very slow.

7. Troubleshooting

Error or symptom	Cause	Fix
ValueError: Invalid CUDA device ... device_count 0	No GPU attached	Session options: set GPU T4/P100, restart, check torch.cuda.is_available()
torch version ends in +cpu	CPU-only PyTorch build	pip install a CUDA build of torch, restart the kernel (Internet On)
Listing the dataset takes forever	os.walk over 20k+ files	Use os.scandir with a depth limit
Scan of labels takes 25 min; 'Slow image access'	Data on /kaggle/input	Copy to /kaggle/working; data.yaml path must be the copy
images not found: .../val/images	Copy incomplete or session wiped	Rerun the resumable copy; check the counts print OK
Read-only file system, 'corrupt image'	Repair attempted in read-only folder	Use the local copy
Weights download fails	Internet is Off	Turn Internet On, or upload yolo11s.pt as a dataset
W&B; login prompt hangs	Weights & Biases integration	!yolo settings wandb=False
Output folder is smoke_fire-3	Earlier runs used the name	exist_ok=True, or locate best.pt with glob
CUDA out of memory	Batch or model too big	batch=16 or 8; smaller model; cache='disk'
Training stops early	patience reached or time cap hit	Expected; check results.csv. Raise patience or time
Page shows 'Canâ€™t', 'Analysingâ€¦'	HTML read with the wrong encoding on Windows	open('index.html', encoding='utf-8')
'Can't reach the detector'	Server not running or wrong address	Start uvicorn; check the address field; check the browser console
Camera blocked	Not https/localhost, or permission denied	Use http://localhost:8000; allow the camera in the address bar

8. Limits and responsible use

- The model knows only smoke and fire. Other objects are ignored or mistaken for them.
- Clouds, fog, sunsets, vehicle lights and orange lamps can trigger false alarms; dim or distant smoke can be missed.
- Results depend on camera angle, distance and lighting compared with the training images.
- **Do not use this as the only fire protection.** Treat it as an extra signal alongside certified smoke alarms and fire systems, and check the dataset's licence before any commercial use.

9. Exercises

- Train yolo11n.pt with the same settings and compare test mAP and speed with yolo11s.pt.
- Change conf from 0.1 to 0.7 on 20 test images and note how false alarms and misses change.
- Add 100 background images with sunsets and lamps (empty label files) and see if false alarms drop.
- Add a third class to data.yaml and label a few images for it.
- Extend main.py with a /detect-video route that returns an alert only after 3 consecutive fire frames.

10. Glossary

Term	Meaning
Bounding box	Rectangle around an object
Epoch	One full pass over the training images
Batch	Images processed together in one training step
Confidence	The model's score (0-1) for a detection
IoU	Intersection over union: overlap between two boxes
mAP	Mean average precision, the main detection metric
NMS	Removes duplicate boxes, keeping the best
Transfer learning	Starting from weights trained on another dataset
Overfitting	Memorizing training data so new data performs worse
Background image	Image with no objects (empty label file)
Checkpoint	Saved model weights (best.pt, last.pt)
Inference	Using a trained model to make predictions
Export	Converting a model to another format (ONNX, TFLite ...)

Final checklist

- GPU shows True before training.
- Data copied to /kaggle/working and every count prints OK.
- data.yaml path is the local copy; class order 0 = smoke, 1 = fire.
- Labels checked visually on at least one image.
- Training finished; best.pt located with glob.
- Test mAP recorded for smoke, fire and all.
- best.pt downloaded next to main.py and index.html.
- Page opened at http://localhost:8000 and tested with a photo.